# OBJECT ORIENTED PROGRAMMING

# [CT-260]

**NAME:** TAHA AHMED MALLICK

**ROLL NO:** CT-25183

**DEPARTMENT:** BCIT **BATCH:** 2025

**YEAR & SECTION:** FSCS-D

**LAB :** 06

# QUESTION#1:

## CODE:

```
#include <iostream>
#include <cmath>
using namespace std;

class Shape
{
private:
    int numSides;
    float area;

public:
    Shape(int sides, float a) : numSides(sides), area(a) {}
    void setSides(int sides) { numSides = sides; }
    void setArea(float a) { area = a; }
    int getSides() { return numSides; }
    float getArea() { return area; }
    void display() {
        cout << "Number of sides: " << numSides << endl;
        cout << "Area: " << area << endl;
    }
};

class Rectangle : public Shape
{
private:
    float len, width;

public:
    Rectangle(float l, float w) : Shape(4, 0), len(l), width(w) {}
    void generateArea() { setArea(len * width); }
    float getLength() { return len; }
    float getWidth() { return width; }
};

class Circle : public Shape
{
private:
    float radius;

public:
```

```
    Circle(float r) : Shape(0, 0), radius(r) {}
    void generateArea() { setArea(M_PI * radius * radius); }
};

class Triangle : public Shape
{
private:
    float base, height;

public:
    Triangle(float b, float h) : Shape(3, 0), base(b), height(h) {}
    void generateArea() { setArea(base * height / 2); }
};

class Square : public Rectangle
{
public:
    Square(float s) : Rectangle(s, s) {}
    void generateArea() { setArea(getLength() * getWidth()); }
    bool checkSide() {
        if (getLength() == getWidth())
            return true;
        return false;
    }
};

int main()
{
    cout << "Rectangle:" << endl;
    Rectangle rect(5, 3);
    rect.generateArea();
    rect.display();

    cout << "\nCircle:" << endl;
    Circle circ(4);
    circ.generateArea();
    circ.display();

    cout << "\nTriangle:" << endl;
    Triangle tri(6, 4);
    tri.generateArea();
    tri.display();

    cout << "\nSquare:" << endl;
    Square sq(4);
```

```
    sq.generateArea();
    sq.display();
    cout << (sq.checkSide() ? "Sides are equal." : "Sides are not equal.") <<
endl;
    return 0;
}
```

# OUTPUT:

```
Rectangle:
Number of sides: 4
Area: 15

Circle:
Number of sides: 0
Area: 50.2655

Triangle:
Number of sides: 3
Area: 12

Square:
Number of sides: 4
Area: 16
Sides are equal.
```

# QUESTION#2:

## CODE:

```
#include <iostream>
using namespace std;

class Calculator
{
private:
    float num1, num2, num3;

public:
    Calculator()
    {
        cout << "Enter three numbers: ";
        cin >> num1 >> num2 >> num3;
    }
    float add(float a, float b)
    {
        return a + b;
    }
    float add(float a, float b, float c)
    {
        return a + b + c;
    }
    float multiply(float a, float b)
    {
        return a * b;
    }
    float multiply(float a, float b, float c)
    {
        return a * b * c;
    }
    float subtract(float a, float b)
    {
        return a - b;
    }
    float divide(float a, float b)
    {
        if (b != 0)
        {
            return a / b;
        }
```

```
        else
        {
            cout << "Error: Division by zero!" << endl;
            return 0;
        }
    }

    void display()
    {
        cout << num1 << " + " << num2 << " = " << add(num1, num2) << endl;
        cout << num1 << " + " << num2 << " + " << num3 << " = " << add(num1,
num2, num3) << endl;
        cout << num1 << " * " << num2 << " = " << multiply(num1, num2) << endl;
        cout << num1 << " * " << num2 << " * " << num3 << " = " << multiply(num1,
num2, num3) << endl;
        cout << num1 << " - " << num2 << " = " << subtract(num1, num2) << endl;
        cout << num1 << " / " << num2 << " = " << divide(num1, num2) << endl;
    }
};

int main()
{
    Calculator calc;
    calc.display();
    return 0;
}
```

# OUTPUT:

```
Enter three numbers: 5
1
2
5 + 1 = 6
5 + 1 + 2 = 8
5 * 1 = 5
5 * 1 * 2 = 10
5 - 1 = 4
5 / 1 = 5
```

# QUESTION#3:

## CODE:

```
#include <iostream>
#include <math.h>
using namespace std;

class Vector {
    float x, y;
public:
    Vector(float x, float y) : x(x), y(y) {}

    Vector operator+(const Vector& other) {
        return Vector(x + other.x, y + other.y);
    }

    Vector operator-(const Vector& other) {
        return Vector(x - other.x, y - other.y);
    }

    Vector operator*(const float val) {
        return Vector(x * val, y * val);
    }

    Vector operator/(const float val) {
        if (val == 0) {
            cout << "Error: Division by zero!" << endl;
            return Vector(0, 0);
        }
        return Vector(x / val, y / val);
    }

    float magnitude() {
        return sqrt(pow(x, 2) + pow(y, 2));
    }

    void display() {
        cout << "Vector(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Vector v1(1, 2), v2(3, 4);
```

```
    cout << "v1: "; v1.display();
    cout << "v2: "; v2.display();

    Vector addition = v1 + v2;
    Vector subtraction = v1 - v2;
    Vector multiplication = v1 * 2;
    Vector division = v1 / 2;
    float mag = v1.magnitude();
    cout << "v1 + v2: "; addition.display();
    cout << "v1 - v2: "; subtraction.display();
    cout << "v1 * 2: "; multiplication.display();
    cout << "v1 / 2: "; division.display();
    cout << "Magnitude of v1: " << mag << endl;
    Vector unitVector = v1 / mag;
    cout << "Unit vector of v1: "; unitVector.display();
    cout << "Magnitude of unit vector: " << unitVector.magnitude() << endl;
    Vector zeroVector = v2 / 0;
    cout << "v2 / 0: "; zeroVector.display();
    return 0;
}
```

# OUTPUT:

```
v1: Vector(1, 2)
v2: Vector(3, 4)
v1 + v2: Vector(4, 6)
v1 - v2: Vector(-2, -2)
v1 * 2: Vector(2, 4)
v1 / 2: Vector(0.5, 1)
Magnitude of v1: 2.23607
Unit vector of v1: Vector(0.447214, 0.894427)
Magnitude of unit vector: 1
Error: Division by zero!
v2 / 0: Vector(0, 0)
```

# QUESTION#4:

## CODE:

```
#include <iostream>
using namespace std;

class Person {
private:
    string name;
    string occupation;
public:
    Person(string name, string occupation) : name(name), occupation(occupation)
{}

    virtual void draw() {
        cout << "A person can draw in many ways" << endl;
    }
};

class Artist : public Person {
public:
    Artist(string name) : Person(name, "Artist") {}

    void draw() override {
        cout << "An artist can draw with a paint brush" << endl;
    }
};

class Gunman : public Person {
public:
    Gunman(string name) : Person(name, "Gunman") {}

    void draw() override {
        cout << "A gunman draws a gun to shoot" << endl;
    }
};

int main() {
    Person *arr[3];
    arr[0] = new Person("ABC", "CS Grad");
    arr[1] = new Artist("XYZ");
    arr[2] = new Gunman("PQR");
    for (int i = 0; i < 3; i++) {
```

```
        arr[i]->draw();
    }
    return 0;
}
```

## OUTPUT:

```
A person can draw in many ways
An artist can draw with a paint brush
A gunman draws a gun to shoot
```